Pandas Tutorial — Lecture 4

This notebook is a **student-friendly tutorial** that matches the topics covered in **Lecture 4: Pandas**.

What you'll learn

- What pandas is and why it's useful
- Series vs DataFrame
- Loading/saving files (CSV, TXT, JSON)
- Inspecting data (`head`, `info`, `describe`, …)
- Selecting data (`[]`, `.loc`, `.iloc`) and the "inclusive vs exclusive" slicing rule
- Cleaning & manipulating (add/drop/rename columns, missing values, sorting)
- Aggregation & grouping (`groupby`)
- Combining DataFrames (`concat`, `merge`)
- Practice exercises (with expected outputs / hints)

> Tip: Run each cell top-to-bottom. Don't skip the exercises 😊

0) Setup

In Colab, most libraries are already installed. If you ever need to install pandas, use `!pip install pandas`.

We'll import pandas (and NumPy for a few examples).

```
import pandas as pd
import numpy as np

pd.__version__
```

```
'2.2.2'
```

1) What is Pandas?

pandas is a Python library for **data manipulation and analysis**.
It builds on top of **NumPy** and adds powerful, easy-to-use **tabular data structures**.

Key idea:

- **Series** = 1D labeled array (like a single column)
- **DataFrame** = 2D labeled table (like a spreadsheet / SQL table)

2) Series (1D)

A **Series** holds:

- values (data)
- an **index** (labels)

When created from a list, pandas creates a default integer index starting at 0.

```
# Series from a list
s_list = pd.Series([10, 20, 30, 40, 50])
s_list
```

	0
0	10
1	20
2	30
3	40
4	50

dtype: int64

Series from a dictionary

When you create a Series from a dictionary, the dictionary **keys become the index**.

```
data = {"Math": 90, "Science": 85, "English": 92}
s_dict = pd.Series(data)
s_dict
```

	0
Math	90
Science	85
English	92

dtype: int64

3) DataFrame (2D)

A **DataFrame** has:

- rows (index)
- columns (labels)
- each column can have a different type (numbers, text, dates, ...)

```
# DataFrame from a dictionary of lists
data = {
    "Student": ["Alice", "Bob", "Charlie"],
    "Math": [90, 85, 92],
    "Science": [88, 91, 80],
}
df_students = pd.DataFrame(data)
df_students
```

	Student	Math	Science
0	Alice	90	88
1	Bob	85	91
2	Charlie	92	80

Next steps: Generate code with `df_students` New interactive sheet

DataFrame from a list of dictionaries (JSON-like)

Useful when you have records like API responses or JSON files.

```
records = [
    {"Name": "Alice", "Age": 24},
    {"Name": "Bob", "Age": 27},
]
```

```
df_records = pd.DataFrame(records)
df_records
```

	Name	Age
0	Alice	24
1	Bob	27

Next steps: Generate code with `df_records` New interactive sheet

4) Loading a dataset (Titanic)

There are 2 easy options in Colab:

1. **Seaborn built-in** dataset (no file needed)
2. Load a CSV from a URL

We'll use a URL so you practice `read_csv`.

```
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titani
df = pd.read_csv(url)

df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs	female	38.0	1	0	PC 17599

Next steps: Generate code with `df` New interactive sheet

5) Initial data inspection

Common commands:

- `df.head(n)` / `df.tail(n)` / `df.sample(n)`

- `df.info()` (types + missing values)
- `df.describe()` (numeric summary)
- `df.shape`, `df.columns`

```
df.head(3)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171
				Cumings,					

Next steps: Generate code with df New interactive sheet

```
df.tail(3)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607

```
df.sample(2, random_state=42)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
709	710	1	3	Moubarek, Master. Halim	male	NaN	1	1	2661

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
```

```
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

```
df.describe()
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	89
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	3
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	4
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	1
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	3
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	51

```
df.shape, df.columns
```

```
((891, 12),
 Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
        'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],
       dtype='object'))
```

6) Selecting columns

- Single column: `df['Age']` → returns a **Series**
- Multiple columns: `df[['Name','Age']]` → returns a **DataFrame** (note the double brackets)

```
age = df["Age"]
type(age), age.head()
```

```
(pandas.core.series.Series,
 0    22.0
 1    38.0
 2    26.0
 3    35.0
 4    35.0
 Name: Age, dtype: float64)
```

```
subset = df[["Name", "Age", "Sex", "Survived"]]
subset.head()
```

	Name	Age	Sex	Survived
0	Braund, Mr. Owen Harris	22.0	male	0
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	38.0	female	1
2	Heikkinen, Miss. Laina	26.0	female	1
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	35.0	female	1
4	Allen, Mr. William Henry	35.0	male	0

Next steps: Generate code with `subset` New interactive sheet

7) `.loc` vs `.iloc` (and "last index inclusiveness")

`.loc` (label-based)

- Uses **labels** (row labels / column names)
- Slicing is **inclusive** of the end label

`.iloc` (position-based)

- Uses **integer positions** (0-based)
- Slicing is like Python: **end is exclusive**

Let's see it.

```
# Using loc with labels (inclusive end)
df.loc[0:2, ["Name", "Age", "Survived"]]
```

	Name	Age	Survived
0	Braund, Mr. Owen Harris	22.0	0
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	38.0	1
2	Heikkinen, Miss. Laina	26.0	1

```
# Using iloc with positions (exclusive end)
df.iloc[0:3, [2, 5, 1]]  # columns by position: Name, Age, Survived (positions
```

	Pclass	Age	Survived
0	3	22.0	0
1	1	38.0	1
2	3	26.0	1

> If you're not sure about column positions, use `df.columns` to check, or just use `.loc` with column names.

8) Conditional filtering

Example: passengers older than 60.

```
df[df["Age"] > 60][["Name", "Age", "Survived"]].head(10)
```

	Name	Age	Survived
33	Wheadon, Mr. Edward H	66.0	0
54	Ostby, Mr. Engelhart Cornelius	65.0	0
96	Goldschmidt, Mr. George B	71.0	0
116	Connors, Mr. Patrick	70.5	0
170	Van der hoef, Mr. Wyckoff	61.0	0
252	Stead, Mr. William Thomas	62.0	0
275	Andrews, Miss. Kornelia Theodosia	63.0	1
280	Duane, Mr. Frank	65.0	0
326	Nysveen, Mr. Johan Hansen	61.0	0
438	Fortune, Mr. Mark	64.0	0

9) Manipulating columns (add / drop)

Add a new column

Example: create `FamilySize` using `SibSp` + `Parch` (siblings/spouses + parents/children).

Drop columns

Use `df.drop(..., axis=1)` to drop columns.

```
df2 = df.copy()
df2["FamilySize"] = df2["SibSp"] + df2["Parch"]
df2[["SibSp", "Parch", "FamilySize"]].head()
```

	SibSp	Parch	FamilySize
0	1	0	1
1	1	0	1
2	0	0	0
3	1	0	1
4	0	0	0

```
# Drop 1 column (returns a NEW DataFrame because inplace=False by default)
df3 = df2.drop("Cabin", axis=1)
df3.columns
```

```
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
       'Parch', 'Ticket', 'Fare', 'Embarked', 'FamilySize'],
      dtype='object')
```

Remember

- `axis=1` targets **columns**
- `axis=0` targets **rows**
- `inplace=True` modifies the original DataFrame (often avoided; many prefer `df = df.drop(...)`)

10) Renaming columns

Use `.rename(columns={old:new})`.

```
df4 = df3.rename(columns={"Pclass": "PassengerClass", "Sex": "Gender"})
df4[["PassengerClass", "Gender"]].head()
```

	PassengerClass	Gender
0	3	male
1	1	female
2	3	female
3	1	female
4	3	male

11) Missing values (NaN)

Steps:

1. Find missing data: `df.isnull().sum()`
2. Drop missing rows/columns: `dropna`
3. Fill missing values (imputation): `fillna`

We'll fill missing `Age` using the mean, then drop rows missing `Embarked`.

```
df4.isnull().sum().sort_values(ascending=False).head(10)
```

	0
Age	177
Embarked	2
PassengerId	0
Survived	0
Name	0
PassengerClass	0
SibSp	0
Gender	0
Parch	0
Ticket	0

dtype: int64

```
df5 = df4.copy()
df5["Age"] = df5["Age"].fillna(df5["Age"].mean())
df5 = df5.dropna(subset=["Embarked"])

df5.isnull().sum().sort_values(ascending=False).head(10)
```

	0
PassengerId	0
Survived	0
PassengerClass	0
Name	0
Gender	0
Age	0
SibSp	0
Parch	0
Ticket	0
Fare	0

dtype: int64

12) Sorting

- `sort_values(by="Age")`
- Multiple columns: `sort_values(by=["PassengerClass","Fare"], ascending=[True, False])`

```
df5.sort_values(by="Age", ascending=True)[["Name", "Age", "Fare"]].head(10)
```

	Name	Age	Fare
803	Thomas, Master. Assad Alexander	0.42	8.5167
755	Hamalainen, Master. Viljo	0.67	14.5000
469	Baclini, Miss. Helene Barbara	0.75	19.2583
644	Baclini, Miss. Eugenie	0.75	19.2583
78	Caldwell, Master. Alden Gates	0.83	29.0000
831	Richards, Master. George Sibley	0.83	18.7500
305	Allison, Master. Hudson Trevor	0.92	151.5500
827	Mallet, Master. Andre	1.00	37.0042
164	Panula, Master. Eino Viljami	1.00	39.6875
788	Dean, Master. Bertram Vere	1.00	20.5750

```
df5.sort_values(by=["PassengerClass", "Fare"], ascending=[True, False])[["Passe
```

	PassengerClass	Name	Fare
258	1	Ward, Miss. Anna	512.3292
679	1	Cardeza, Mr. Thomas Drake Martinez	512.3292
737	1	Lesurer, Mr. Gustave J	512.3292
27	1	Fortune, Mr. Charles Alexander	263.0000
88	1	Fortune, Miss. Mabel Helen	263.0000
341	1	Fortune, Miss. Alice Elizabeth	263.0000
438	1	Fortune, Mr. Mark	263.0000
311	1	Ryerson, Miss. Emily Borie	262.3750
742	1	Ryerson, Miss. Susan Parker "Suzette"	262.3750
118	1	Baxter, Mr. Quigg Edmond	247.5208

13) Aggregation (basic stats)

Common methods:

- `.mean()`, `.median()`, `.sum()`, `.count()`, `.min()`, `.max()`
- `.value_counts()`

```
df5["Fare"].mean(), df5["Fare"].median(), df5["Fare"].max()
```

```
(np.float64(32.09668087739032), 14.4542, 512.3292)
```

```
df5["Embarked"].value_counts()
```

Embarked	count
S	644
C	168
Q	77

dtype: int64

14) GroupBy (Split–Apply–Combine)

Example: mean survival rate by passenger class.

```
df5.groupby("PassengerClass")["Survived"].mean().sort_index()
```

PassengerClass	Survived
1	0.626168
2	0.472826
3	0.242363

dtype: float64

Group by multiple columns

Example: survival rate by class and gender.

```
df5.groupby(["PassengerClass", "Gender"])["Survived"].mean()
```

PassengerClass	Gender	Survived
1	female	0.967391
	male	0.368852
2	female	0.921053
	male	0.157407
3	female	0.500000
	male	0.135447

dtype: float64

15) Combining DataFrames

`pd.concat`

- `axis=0`: stack rows (one on top of another)
- `axis=1`: stack columns side-by-side (align on index)

`pd.merge`

SQL-style joins on a key (like `StudentID`).

```
df_a = pd.DataFrame({"id":[1,2,3], "score":[80, 90, 75]})
df_b = pd.DataFrame({"id":[4,5], "score":[88, 92]})

pd.concat([df_a, df_b], axis=0, ignore_index=True)
```

	id	score
0	1	80
1	2	90
2	3	75
3	4	88
4	5	92

```
students = pd.DataFrame({
    "StudentID":[1,2,3],
```

```
    "Name":["Alice","Bob","Charlie"]
})
courses = pd.DataFrame({
    "StudentID":[1,1,3],
    "Course":["Math","Science","History"]
})

pd.merge(students, courses, on="StudentID", how="inner")
```

	StudentID	Name	Course
0	1	Alice	Math
1	1	Alice	Science
2	3	Charlie	History

16) Saving data (CSV / TXT / JSON)

CSV

- `index=False` usually avoids exporting the extra index column.

TXT (space-separated)

- Use `sep=' '`.

JSON

- `orient="records"` is a very common, clean layout (list of row objects).

```
# We'll save a small sample so you don't download a huge file by accident.
sample = df5[["Name", "Age", "Fare", "Survived", "PassengerClass", "Gender"]].h

sample.to_csv("diabetes_out.csv", index=False)
sample.to_csv("diabetes_out.txt", index=False, sep=" ")
sample.to_json("diabetes_out.json", orient="records", indent=2)

# Show files created in Colab
import os
[f for f in os.listdir(".") if f.startswith("diabetes_out")]
```

```
['diabetes_out.csv', 'diabetes_out.txt', 'diabetes_out.json']
```

✅ Practice Exercises

Try these exercises. Write your code in the empty cells and run them.

Exercise 1

Load the Titanic dataset again into `df_ex` and print:

- `df_ex.shape`
- first 5 rows

Exercise 2

Using `.loc`, select rows **0 to 3** (inclusive) and columns `Name`, `Age`, `Survived`.

Exercise 3

Using `.iloc`, select the **first 4 rows** and the **first 3 columns**.

Exercise 4

Create a new column `IsChild` where `Age < 18`.

Exercise 5

Compute the survival rate (mean of `Survived`) by `Sex`.

Exercise 6 (Challenge)

Find the **top 10 highest fares** among passengers with:

- `PassengerClass == 1`
- `Age` not missing Show `Name`, `Age`, `Fare`.

Hints

- Use `df.isnull().sum()` to check missing values.
- Use `sort_values(by=..., ascending=...)` for ranking.

```
# Exercise 1 (your code)
import pandas as pd
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titani
df_ex = pd.read_csv(url)
print(df_ex.shape)
df_ex.head()
```

```
(891, 12)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs	female	38.0	1	0	PC 17599

Next steps: Generate code with df_ex New interactive sheet

```
# Exercise 2 (your code)
df_ex.loc[0:3, ["Name", "Age", "Survived"]]
```

	Name	Age	Survived
0	Braund, Mr. Owen Harris	22.0	0
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	38.0	1
2	Heikkinen, Miss. Laina	26.0	1
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	35.0	1